

核心设计原则

关注点分离 (Separation of Concerns): 将模拟器的不同职责严格分开。一个模拟任务可以分解为: 用什么协议 (Protocol) -> 发什么格式 (Codec) -> 发什么内容 (Data) -> 按什么逻辑发 (Strategy)。

面向接口编程 (Interface-Based Design): 为每个职责定义清晰的 Java 接口。所有具体实现都将实现这些接口。

配置驱动 (Configuration-Driven): 模拟器的所有行为 (模拟哪个设备、使用哪个协议、数据格式等) 都由外部配置文件 (如 JSON) 定义, 无需修改代码。

工厂与反射 (Factories & Reflection): 框架的核心引擎根据配置文件中的类型名称, 动态地创建和组装相应的实现类的实例。

核心层:

配置层 (Configuration Layer)

这是驱动一切的起点。我们设计一个主配置文件, 它可以定义一个或多个“模拟画像 (Simulation Profile)”。每个画像代表一个要模拟的设备。

示例: mindray-monitor-profile.json (这是我们之前特定实现的“通用版”)

```
1 {
2   "profileName": "Mindray Monitor HL7",
3   "enabled": true,
4   "device": {
5     "id": "Sim-Mindray-001",
6     "description": "模拟迈瑞监护仪, 通过 TCP 发送 HL7 v2.6 ORU 消息"
7   },
8   "protocol": {
9     "type": "tcp-client",
10    "host": "127.0.0.1",
11    "port": 18888,
12    "reconnectDelay": 5000
13  },
14  "codec": {
15    "type": "hl7",
16    "version": "2.6",
17    "mllpFrame": true,
18    "charset": "UTF-8"
19  },
20  "strategy": {
21    "type": "periodic-push",
22    "intervalMillis": 5000,
23    "dataGenerator": {
24      "type": "random",
25      "properties": [
26        { "key": "MDC_ECG_HEART_RATE", "dataType": "int", "min": 60, "max": 120 },
```

```

27     { "key": "MDC_PULS_OXIM_SAT_O2", "dataType": "int", "min": 95, "max": 100 }
28   ]
29 }
30 }
31 }

```

示例: infusion-pump-profile.json (一个假想的、使用 JSON 通过 HTTP 上报的输液泵)

```

1 {
2   "profileName": "Zhongdao Infusion Pump JSON",
3   "enabled": true,
4   "device": {
5     "id": "ZD-Pump-SN002"
6   },
7   "protocol": {
8     "type": "http-post",
9     "url": "http://localhost:8080/api/device-data",
10    "headers": {
11      "Authorization": "Bearer some-token"
12    }
13  },
14  "codec": {
15    "type": "json"
16  },
17  "strategy": {
18    "type": "periodic-push",
19    "intervalMillis": 10000,
20    "dataGenerator": {
21      "type": "random",
22      "properties": [
23        { "key": "flowRate", "dataType": "double", "min": 5.0, "max": 20.0, "precision": 1 },
24        { "key": "remainingVolume", "dataType": "double", "min": 0, "max": 500,
"precision": 0 },
25        { "key": "status", "dataType": "enum", "values": ["running", "paused", "empty"] }
26      ]
27    }
28  }
29 }

```

核心引擎 (Core Engine)

这是框架的大脑，负责解析配置、动态加载和管理各个模拟画像的生命周期。

SimulationManager:

启动时扫描 `simulation-profiles` 目录，加载所有 `*.json` 文件。

为每个 `enabled: true` 的画像创建一个 `SimulationInstance`。

在独立的线程中启动每个 `SimulationInstance`。

负责优雅地停止所有模拟。

SimulationInstance:

持有一个画像的完整配置。

使用工厂类，根据配置中的 `type` 字段（如`"protocol": {"type": "tcp-client"}`）创建对应的 `ProtocolHandler`, `MessageCodec` 和 `SimulationStrategy` 实例。

调用 `SimulationStrategy` 的 `execute()` 方法来启动模拟流程。

协议层 (ProtocolHandler Interface)

职责: 处理网络通信的底层细节，只负责发送和接收字节流 (`byte[]`)。

`com.xinsec.simulator.protocols.ProtocolHandler.java`:

```
1 public interface ProtocolHandler {
2     void configure(Config config); // 传入 protocol 部分的配置
3     void start(MessageListener listener); // 启动连接，并设置一个监听器来接收数据
4     void stop();
5     void send(byte[] data);
6
7     interface MessageListener {
8         void onMessageReceived(byte[] data);
9     }
10 }
```

实现类:

`TcpClientHandler`: 实现 TCP 客户端连接、断线重连。

`UdpHandler`: 实现 UDP 通信。

`HttpPostHandler`: 实现 HTTP POST 请求。

`TcpServerHandler`: 模拟一个需要设备主动来连接的服务端。

编解码层 (MessageCodec Interface)

职责: 在结构化数据（如 `Map<String, Object>`）和字节流 (`byte[]`) 之间进行转换。

`com.xinsec.simulator.codecs.MessageCodec.java`:

```
1 public interface MessageCodec {
2     void configure(Config config); // 传入 codec 部分的配置
3     byte[] encode(Map<String, Object> data, Map<String, Object> metadata); // 将业务数据
    编码为字节
4     Map<String, Object> decode(byte[] rawData); // 将收到的字节解码为业务数据
5 }
```

实现类:

`HL7Codec`: 将 `Map` 数据构建成 HL7 字符串，并根据配置添加 MLLP 帧。

`JsonCodec`: 使用 Jackson 库将 `Map` 序列化为 JSON 字符串字节。

`FixedLengthBinaryCodec`: 按固定长度和偏移量将数据编码为二进制报文。

`DelimiterBasedCodec`: 基于分隔符的文本协议编解码器。

数据生成层 (DataGenerator Interface)

职责: 创建模拟的业务数据。

com.xinsec.simulator.datagen.DataGenerator.java:

```
1 public interface DataGenerator {
2     void configure(Config config); // 传入 dataGenerator 部分的配置
3     Map<String, Object> generate(); // 生成一组模拟数据
4 }
```

实现类:

RandomDataGenerator: 根据配置的 min/max/precision/enum 等规则生成随机数据。

CsvReplayDataGenerator: 从 CSV 文件中顺序读取数据, 用于回放历史场景。

SinusoidalDataGenerator: 生成符合正弦/余弦曲线的模拟数据 (如模拟呼吸波形)。

策略/工作流层 (SimulationStrategy Interface)

职责: 编排以上所有组件, 执行具体的业务模拟逻辑。

com.xinsec.simulator.strategies.SimulationStrategy.java:

```
1 public interface SimulationStrategy {
2     void configure(Config config, ProtocolHandler protocol, MessageCodec codec,
3     DataGenerator generator);
4     void execute(); // 启动策略
5     void stop();
6 }
```

实现类:

PeriodicPushStrategy: 最常见的策略。使用 ScheduledExecutorService 定时调用 DataGenerator -> MessageCodec -> ProtocolHandler.send()。

RequestResponseStrategy: 监听 ProtocolHandler 的 onMessageReceived 事件, 收到消息后, 调用 MessageCodec.decode(), 可能触发 DataGenerator 生成响应数据, 再 encode 并 send

StatefulHandshakeStrategy: 一个更复杂的策略, 例如: 先发送一个登录包, 等待一个特定的响应, 成功后再切换到 PeriodicPushStrategy 的逻辑。

如何扩展

假设需要支持一个新的、基于 WebSocket 的设备:

实现 WebSocketHandler: 创建一个实现 ProtocolHandler 接口的类, 处理 WebSocket 连接逻辑。

实现 MyCustomXmlCodec: 如果它使用 XML, 就创建一个实现 MessageCodec 的类来处理 XML 的构建和解析。

编写 my-websocket-device.json 配置文件:

```
1 {
2     "profileName": "My WebSocket Device",
3     "protocol": { "type": "websocket", "uri": "ws://localhost:9000/ws" },
4     "codec": { "type": "my-custom-xml" },
5     "strategy": { ... }
6 }
```

4. 启动模拟器: 核心引擎会自动发现新的 type 名称, 并通过工厂创建你的新实现类。无需修改任何框架核心代码。

项目结构建议(仅供参考,根据实际情况做修改)

```
1 device-simulator/
2   └─ src/
3       └─ main/
4           └─ java/com/xinsec/simulator/
5               │   └─ Main.java           # 程序入口
6               │   └─ core/
7                   │   └─ SimulationManager.java
8                   │   └─ SimulationInstance.java
9                   └─ ComponentFactory.java # 动态创建组件的工厂
10              └─ config/                  # 配置相关的 POJO 类
11                  └─ protocols/
12                      │   └─ ProtocolHandler.java # 接口
13                      └─ impl/
14                          └─ TcpClientHandler.java
15                          └─ HttpPostHandler.java
16                  └─ codecs/
17                      │   └─ MessageCodec.java    # 接口
18                      └─ impl/
19                          └─ Hl7Codec.java
20                          └─ JsonCodec.java
21                  └─ datagen/
22                      │   └─ DataGenerator.java    # 接口
23                      └─ impl/
24                          └─ RandomDataGenerator.java
25                  └─ strategies/
26                      │   └─ SimulationStrategy.java # 接口
27                      └─ impl/
28                          └─ PeriodicPushStrategy.java
29              └─ resources/
30                  └─ logback.xml
31                  └─ simulation-profiles/
32                      └─ mindray-monitor-profile.json
33                      └─ infusion-pump-profile.json
```

扩展 1: 支持原报文测试 (Raw Message Replay)

这个功能允许测试人员直接使用捕获到的、未经修改的原始报文进行测试,对于复现特定问题或进行边界测试至关重要。

我们通过引入一个新的 `SimulationStrategy` 来实现这一点,而不是修改现有组件,这完全符合我们的设计原则。

新增策略: `RawReplayStrategy`

职责: 此策略不使用 `DataGenerator` 和 `MessageCodec`。它的唯一工作就是按配置读

取一个或多个原始报文，并直接通过 `ProtocolHandler` 发送出去。

更新配置层

我们需要在画像配置文件中支持这种新策略。

示例: `raw-hl7-replay-profile.json`

```
{
  "profileName": "HL7 Raw Message Replay",
  "enabled": true,
  "device": {
    "id": "Replay-Device-01"
  },
  "protocol": {
    "type": "tcp-client",
    "host": "127.0.0.1",
    "port": 18888
  },
  "codec": {
    "type": "passthrough"
  },
  "strategy": {
    "type": "raw-replay",
    "intervalMillis": 3000,
    "replayMode": "sequential",
    "messages": [
      {
        "encoding": "hex",
        "data":
"0B4D53487C5E7E5C267C4D494E445241595F4E2D5345524945535E3030413033373030393930
30343738335E4555492D36347C7C7C7C32303231303131343136353134393030302B303030307
C7C4F52555E5230315E4F52555F5230317C3230397C507C322E360D4F42587C317C4E4D7C3134
373834325E4D44435F4543475F48454152545F524154455E4D44437C7C39357C7C7C44454D4F1
C0D"
      },
      {
        "encoding": "base64",
        "data":
"C01TSHxbXlx&fE1JTkRSQVlfti1TRVJJRVNeMDBBMDM3MDA5OTAwNDc4M15FVUktNjR8fHx8Mj
AyMTAxMTQxNjUxNDkwMDArMDAwMHx8T1JVXllwMV5PUlVfUjAxHlwoXxQfDIuNgOKTOJYDF8
Tk18MTQ3ODQyXk1EQ19FQ0dfSEVBUIRfUkFURV5NREN8fDk4fHx8REVNTx0N"
      },
      {
        "encoding": "utf8",
        "data": "\u000bMSH|^~\&|||2024...|ORU^R01|... \u001c\r"
      }
    ]
  }
}
```

```
}
}
```

strategy.type: 设置为 raw-replay。

codec.type: 可以设置为 passthrough (直通), 表示编解码层不做任何操作。

replayMode: 播放模式, 可以是 sequential (顺序循环播放) 或 random (随机选择一个播放)。

messages: 一个数组, 包含要发送的原始报文。

encoding: 报文数据的编码格式, 支持 hex, base64, utf8 (普通字符串), 以便于在 JSON 中存储和读取二进制数据。

data: 编码后的报文数据。

RawReplayStrategy 在执行时, 会解析 messages 数组, 根据 encoding 将 data 还原为 byte[], 然后定时发送。

扩展 2: 支持数据库配置

将配置从文件系统移到数据库, 是实现配置的集中化、动态化管理和支持 UI 操作的基础。我们将通过抽象 配置提供者 (Configuration Provider) 来实现。

新增抽象层: ProfileProvider

职责: 负责加载一个或多个模拟画像配置。这是连接核心引擎和配置源的桥梁。

com.xinsec.simulator.provider.ProfileProvider.java:

```
public interface ProfileProvider {
    /**
     * 加载所有可用的模拟画像配置
     * @return 配置对象列表
     */
    List<SimulationProfile> loadProfiles();
}
```

提供两种 ProfileProvider 实现

FileProfileProvider (默认实现):

实现 ProfileProvider 接口。

其 loadProfiles() 方法的逻辑就是我们之前设计的: 扫描 simulation-profiles 目录, 解析所有 JSON 文件。

DatabaseProfileProvider (新实现):

同样实现 ProfileProvider 接口。

它将连接数据库来获取配置。

数据库表设计

为了最大的灵活性, 我们不建议使用复杂的范式将 JSON 配置拆分到多个表中。最佳实践是使用一个 TEXT 或 JSON 类型的字段来存储整个画像的 JSON 配置。

表名: simulation_profiles

列名	类型	描述
id	BIGINT / UUID	主键

profile_name	VARCHAR(255)	画像名称，用于 UI 显示和管理
is_enabled	BOOLEAN	是否启用此画像，引擎将只加载启用的
profile_config	TEXT / JSON	存储完整画像配置的 JSON 字符串
created_at	TIMESTAMP	创建时间
updated_at	TIMESTAMP	最后更新时间

DatabaseProfileProvider 的 loadProfiles() 方法会查询此表，取出 profile_config 字段的 JSON 字符串，然后使用 Jackson 等库反序列化为 SimulationProfile 配置对象。

框架自身的配置

现在，模拟器框架本身需要一个配置文件，来决定使用哪个 ProfileProvider 以及如何连接数据库。

src/main/resources/application.yml:

```

simulator:
  config:
    # 配置源类型: "file" 或 "database"
    provider: "database"

# 仅当 provider = "database" 时需要此部分
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/simulator_db?useUnicode=true&characterEncoding=utf8
    username: root
    password: your_password
    driver-class-name: com.mysql.cj.jdbc.Driver
  jpa:
    hibernate:
      ddl-auto: update

```

核心引擎在启动时，会首先读取 application.yml，根据 simulator.config.provider 的值，通过工厂模式创建 FileProfileProvider 或 DatabaseProfileProvider 的实例，然后用它来加载所有模拟画像。

架构更新总结

通过以上扩展：

策略层 (Strategy): 新增了 RawReplayStrategy，与 PeriodicPushStrategy 等并存，提供了“生成模拟数据”和“回放原始数据”两种能力。

配置层 (Configuration): 变得可插拔。通过 ProfileProvider 接口，将配置的 存储方式 (文件/数据库) 与配置的 使用者 (核心引擎) 完全解耦。

企业级特性: 支持数据库配置和动态管理，为未来构建 Web 管理界面（如在线启停、编辑模拟画像）打下了坚实的基础。

